Who submitted the workload?

How to reproduce the tests and what is still missing | 16 October 2026

Enable it in the exporter

The PoC adds `submitted_by` to `karta_pod_workload_info` and `karta_workload_status`. Karta's CRD and SDK stay unchanged. Admission captures direct Kubernetes submissions. CEL reads the metadata supplied by each platform.

```
exporter:
  enabled: true
  useCatalog: true
  attribution:
    admission:
      enabled: true
    submittedByExpression: |
      (
        [
          "workload.dgxc.nvidia.com/submitter-id",
          "run.ai/submitted-by",
          "workflow.example.com/submitter",
          "platform.example.com/username",
          "platform.example.com/created-by"
        ]
        .filter(key, key in annotations && annotations[key] != "")
        .map(key, annotations[key])
        +
        [
          "workload.dgxc.nvidia.com/user",
          "identity.example.com/submitted-by",
          "osmo.submitted_by",
          "workflow.example.com/submitter",
          "platform.example.com/username",
          "platform.example.com/created-by"
        ]
        .filter(key, key in labels && labels[key] != "")
        .map(key, labels[key])
        + [""]
      )[0]
```

Admission records the authenticated Kubernetes caller before a selected workload is saved. Karta's operator serves this webhook over HTTPS (TLS). Within the selected scope, it also rejects changes to the recorded identity. Keep `operator.enabled` and `webhook.enabled` true. Setting `exporter.attribution.admission.enabled: false` stops capture; CEL can still read existing platform metadata.

The full expression above was tested. A workload label also supplies the identity on its Pod metric. An empty `submitted_by` means unknown; it is not a claim that no user exists. NVIDIA username and organization mapping still need agreement. The organization example below uses a test annotation; no organization metric was added.

Choose where admission captures the caller

Already supported: namespace include/exclude selectors and a resource allowlist for admission. These settings do not filter exporter watches. Add this under the existing exporter values; keep `submittedByExpression` unchanged.

```
exporter:
  attribution:
    admission:
      enabled: true
      namespaceSelector:
        matchExpressions:
          - key: karta.run.ai/capture-submitter
            operator: In
            values: ["true"]
          - key: kubernetes.io/metadata.name
            operator: NotIn
            values: [kube-system, karta-system]
      resourceRules:
        - apiGroups: ["apps"]
          apiVersions: ["v1"]
          resources: ["deployments", "statefulsets"]
        - apiGroups: ["batch"]
          apiVersions: ["v1"]
          resources: ["jobs", "cronjobs"]
        - apiGroups: [""]
          apiVersions: ["v1"]
          resources: ["pods"]
```

Include namespaces labeled `karta.run.ai/capture-submitter=true`; exclude the two named namespaces even if labeled. Both expressions must match. To include namespace names, use `kubernetes.io/metadata.name` with `In`.

Resource rules use API group/version and plural resource names: `apps/v1/deployments` selects Deployment. Today, exclude a type by removing it from this allowlist. Replace `karta-system` with the operator's namespace.

Proposed: match group, version and kind (GVK)

The chart does not expose `matchConditions` yet. Proposed addition under the same admission block, applied to both capture and update protection:

```
exporter:
  attribution:
    admission:
      matchConditions: # Proposed; not implemented in the chart
        - name: include-gvks
          expression: |
            request.kind.version == "v1" && (
              (request.kind.group == "apps" &&
               request.kind.kind in ["Deployment", "StatefulSet"]) ||
              (request.kind.group == "batch" &&
               request.kind.kind in ["Job", "CronJob"]) ||
              (request.kind.group == "" && request.kind.kind == "Pod"))
        - name: exclude-gvks
          expression: |
            !(request.kind.group == "batch" &&
              request.kind.version == "v1" &&
              request.kind.kind == "CronJob")
```

All conditions must pass, so the CronJob exclusion wins. Excluded requests skip submitter capture and update protection. `submittedByExpression` separately selects the identity to export. Namespace labels and scope settings must remain administrator-controlled.

This follows Kubernetes webhook matching. Only `matchConditions` is proposed; namespace selectors and resource allowlists already work.

Match labels or annotations for attribution

The current PoC supports CEL `rules[].matchExpression`. This chooses where a rule extracts identity; it does not remove workloads from metrics or reduce Kubernetes LIST/WATCH traffic. Use this alternative to the first page's expression. Clear `submittedByExpression` when switching to `rules`.

```
exporter:
  enabled: true
  attribution:
    enabled: true
    submittedByExpression: ""
    rules:
      - name: configured-submitter
        matchExpression: |
          ("workload.dgxc.nvidia.com/user" in labels &&
           labels["workload.dgxc.nvidia.com/user"] != "") ||
          ("workload.dgxc.nvidia.com/submitter-id" in annotations &&
           annotations["workload.dgxc.nvidia.com/submitter-id"] != "")
        submittedByExpression: |
          ("workload.dgxc.nvidia.com/submitter-id" in annotations &&
           annotations["workload.dgxc.nvidia.com/submitter-id"] != "")
            ? annotations["workload.dgxc.nvidia.com/submitter-id"]
            : labels["workload.dgxc.nvidia.com/user"]
        identitySourceExpression: "'configured-metadata'"
        submitterTypeExpression: "'unknown'"
        partial: true
```

Either nonempty key matches. The annotation wins when both exist on the same object. Without either key, this rule supplies no identity; a Pod can still inherit identity from its workload. A metadata update triggers reevaluation. Changing the configured expression requires an exporter rollout.

The chart and evaluator checks cover this exact example. It is an alternative configuration, not the configuration used for the platform captures below. The first page's full expression also accepts native Run:AI and OSMO keys.

Watch filtering with a Helm `workloadSelector.matchExpressions` is still proposed, not implemented. Kubernetes supports label selectors on LIST/WATCH; annotations need filtering inside the exporter. Native selector requirements use AND; the CEL example above uses OR.

Any watch-filter implementation must retain required child Pods and intermediate owners even when they have no matching label. Matching only root labels is not enough to resolve the ownership graph. A selector also does not discover arbitrary workload kinds or grant RBAC.

Direct Kubernetes: capture and protect the caller

As an administrator, create `karta-label-direct` and label it `karta.run.ai/capture-submitter=true`. Use a user's kubeconfig with get/list/watch/create/patch permission there. The test used a one-hour certificate for `alice@example.test`, without impersonation.

Save as `direct-job.yaml`:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: cel-label-job
  namespace: karta-label-direct
  annotations: {review.example.com/note: keep-me}
spec:
  backoffLimit: 0
  ttlSecondsAfterFinished: 3600
  template:
    spec:
      automountServiceAccountToken: false
      restartPolicy: Never
      containers:
      - name: work
        image: busybox:1.37.0
        command: [sh, -c, "echo submitter-demo; sleep 300"]
        resources:
          requests: {cpu: 5m, memory: 8Mi}
          limits: {cpu: 100m, memory: 32Mi}
```

```
kubectl --kubeconfig "$USER_KUBECONFIG" auth whoami
kubectl --kubeconfig "$USER_KUBECONFIG" apply -f direct-job.yaml
```

Admission wrote `metadata.annotations["workload.dgxc.nvidia.com/submitter-id"]` on the Job as `alice@example.test`. CEL reads that annotation; the Pod metric inherits it from the Job. The Pod needs no copied annotation. The Job completed after 300 seconds; a separate Deployment also ran successfully. Selected metric labels:

```
karta_workload_status{workload="cel-label-job",
  phase="Completed", submitted_by="alice@example.test"} 1
karta_pod_workload_info{workload="cel-label-job",
  submitted_by="alice@example.test"} 1
```

Try claiming another user at creation, then changing the saved identity:

```
KEY=workload.dgxc.nvidia.com/submitter-id
kubectl --kubeconfig "$USER_KUBECONFIG" create \
  --dry-run=client -f direct-job.yaml -o json |
  jq --arg key "$KEY" '.metadata.name="cel-label-forged" |
    .metadata.annotations[$key]="bob@example.test"' |
  kubectl --kubeconfig "$USER_KUBECONFIG" create -f -
kubectl --kubeconfig "$USER_KUBECONFIG" -n karta-label-direct \
  annotate job cel-label-job "$KEY=bob@example.test" --overwrite
```

Both were rejected. A caller may prefill their own matching identity. Unrelated metadata is preserved. Protection applies only to the configured scope. Creation by a system caller is skipped when the object has owner references. A service account alone does not identify the human behind it.

Captured requests and rejection results.

Run:AI: submit training and inference

Use a token with submission permission and set `RUNAI_API`, `RUNAI_TOKEN`, `PROJECT_ID` and `CLUSTER_ID`. The test used the Run:AI password login flow. These commands contain the full requests; no request files need to be downloaded.

```
jq -n --arg p "$PROJECT_ID" --arg c "$CLUSTER_ID" '{
  name: "cel-label-training", projectId: $p, clusterId: $c,
  spec: {
    image: "busybox:1.37.0", command: "sh",
    args: "-c \"echo attribution-start; sleep 300; echo done\"",
    backoffLimit: 0, restartPolicy: "Never",
    compute: {
      cpuCoreRequest: 0.05, cpuCoreLimit: 0.2,
      cpuMemoryRequest: "64Mi", cpuMemoryLimit: "128Mi",
      gpuDevicesRequest: 0
    }
  }
}' > training.json

jq '
  .name="cel-label-inference" |
  del(.spec.backoffLimit, .spec.restartPolicy) |
  .spec.args=("-c \"mkdir -p /tmp/www; " +
    "echo attribution-inference > /tmp/www/index.html; " +
    "httpd -f -p 8080 -h /tmp/www\"") |
  .spec.autoscaling={minReplicas:1,maxReplicas:1} |
  .spec.servingPort={
    container:8080,protocol:"http",clusterLocalAccessOnly:true}
' training.json > inference.json

for kind in training inference; do
  curl -fsS "$RUNAI_API/api/v1/workloads/${kind}s" \
    -H "Authorization: Bearer $RUNAI_TOKEN" \
    -H 'Content-Type: application/json' --data-binary @"$kind.json"
done
```

Training completed after 300 seconds. Inference served HTTP 200 on port 8080 with one replica. These CPU-only commands exercise submission and attribution, not model inference.

Required setup for this test:

- A Run:AI project and enough CPU capacity for both workloads.
- Knative serving for this inference path. Its `kubernetes.podspec-schedulername` and `kubernetes.podspec-affinity` flags allowed the scheduler and affinity settings; they do not enable identity extraction.
- Exporter RBAC with `get/list/watch` on `run.ai/workloadrunners`, so Karta can inspect the ownership graph. Keep the chart-default access to Jobs, Pods, apps Deployments/ReplicaSets and Knative Services/Configurations/Revisions. Add a ClusterRole labeled `karta.run.ai/exporter-rbac: "true"` for WorkloadRunners; the chart aggregates it without replacing defaults.

Run:AI: where the identity appears

Run:AI supplied the identity on these objects. Captured fields:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: cel-label-training
  annotations:
    run.ai/submitted-by: test@run.ai
---
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: cel-label-inference
  annotations:
    run.ai/submitted-by: test@run.ai
```

CEL reads `annotations["run.ai/submitted-by"]`. The WorkloadRunner CRs had no submitter annotation. Karta attributed these Pods to the Job and Service above. The inference Pod had no submitter key; its metric inherited the Service's value.

```
karta_workload_status{
  workload="cel-label-training", phase="Completed",
  submitted_by="test@run.ai"
} 1
karta_pod_workload_info{
  workload="cel-label-inference", submitted_by="test@run.ai"
} 1
```

The same CEL reads the annotation on both the Job and the Knative Service. No external identity mapper was used. The separate exporter was for lab isolation.

Set `RUNAI_NS` to the project's Kubernetes namespace. Use cluster credentials with read and Pod port-forward access. Once the objects appear, repeat the completion/readiness and HTTP checks:

```
kubectl -n "$RUNAI_NS" wait --for=condition=complete \
  job/cel-label-training --timeout=10m
kubectl -n "$RUNAI_NS" wait --for=condition=Ready \
  services.serving.knative.dev/cel-label-inference --timeout=10m
POD=$(kubectl -n "$RUNAI_NS" get pods \
  -l serving.knative.dev/service=cel-label-inference \
  -o jsonpath='{.items[0].metadata.name}')
kubectl -n "$RUNAI_NS" port-forward "pod/$POD" 18080:8012
```

In another terminal:

```
curl -i http://127.0.0.1:18080/
```

Expect HTTP 200 and `attribution-inference`. Stop the port-forward with Ctrl-C. This checks the Knative queue proxy; external ingress was not tested.

Current gap: agree the NVIDIA username format with Run:AI. No submitter propagation bug was observed in these two paths.

See the captured resource fields and Run:AI metrics.

OSMO

Use a logged-in OSMO CLI. Replace `YOUR_CPU_PLATFORM` and `YOUR_POOL` with your platform and pool. Each block creates and submits the complete workflow; no files to download.

Single task:

```
cat > osmo-single.yaml <<'YAML'
workflow:
  name: cel-label-single
  resources:
    default:
      cpu: 1
      memory: 512Mi
      storage: 1Gi
      platform: YOUR_CPU_PLATFORM
  tasks:
  - name: worker
    image: ubuntu:24.04
    command: [/bin/sh, -c]
    args:
    - echo attribution-start; sleep 300; echo attribution-complete
YAML
osmo workflow submit osmo-single.yaml --pool YOUR_POOL
```

Leader/worker group:

```
cat > osmo-group.yaml <<'YAML'
workflow:
  name: cel-label-group
  resources:
    default:
      cpu: 1
      memory: 512Mi
      storage: 1Gi
      platform: YOUR_CPU_PLATFORM
  groups:
  - name: pair
    tasks:
    - name: leader
      image: ubuntu:24.04
      command: [/bin/sh, -c]
      args:
      - echo attribution-start; sleep 300; echo attribution-complete
      lead: true
    - name: worker
      image: ubuntu:24.04
      command: [/bin/sh, -c]
      args:
      - echo attribution-start; sleep 300; echo attribution-complete
YAML
osmo workflow submit osmo-group.yaml --pool YOUR_POOL
```

Use the workflow ID returned by each command to check completion:

```
osmo workflow query YOUR_WORKFLOW_ID
```

These are the same workflow specifications used in the API test below. The CLI commands are the documented reproduction path; the saved results came from REST submissions.

OSMO: result and existing options

OSMO 6.3.1 already supplies `osmo.submitted_by`. In the single-task and leader/worker tests, the API returned `submitted_by: alice@example.test`; task Pods and KAI PodGroups carried `osmo.submitted_by: alice`. Both workflows completed. In the repeated run, all three task exit codes were 0.

An administrator can add the agreed username label without changing OSMO code. Register this Pod template and append its name to the selected platform's `override_pod_template` list. Keep the existing templates.

```
metadata:
  labels:
    workload.dgxc.nvidia.com/user: "{{WF_SUBMITTED_BY | default('')}}"
    osmo.submitted_by: "{{WF_SUBMITTED_BY | default('')}}"
```

In 6.3.1 database configuration mode, create the named template through `PUT /api/configs/pod_template/attribution` with `{"configs": <template>}`; use an administrator credential. In ConfigMap mode, manage it through Helm/GitOps. The 6.3.1 service chart uses `services.configs.podTemplates` and `services.configs.pools`. Version 6.4 is ConfigMap-only: use `configuration.podTemplates` and `configuration.pools`. If `configuration.snapshot` is set, edit the equivalent fields inside that snapshot instead. Do not change configuration mode for this example.

Captured Pod fields, with unrelated metadata omitted:

```
metadata:
  labels:
    osmo.workflow_id: suggestion-shared-email-1
    osmo.task_name: worker
    osmo.submitted_by: alice
    workload.dgxc.nvidia.com/user: alice
```

```
karta_pod_workload_info{
  workload="21c6fd9fd8db4da8-bb22bddc61ac4673",
  workload_kind="Pod", submitted_by="alice"
} 1
```

The same value appeared on `karta_workload_status`. This is sufficient if `alice` is the agreed unique username. It is not a directory lookup. The template variable has already lost the email domain; `alice@example.test` and `alice@another.example` cannot be distinguished from that value alone.

Current edge case: the user `a` has no native submitter label. An unguarded `{{WF_SUBMITTED_BY}}` template failed; the workflow stayed PENDING until it was cancelled. The guard above keeps missing attribution empty. The one-character omission and raw-principal limitation are included in the OSMO draft.

Version note: workflow `labels` / CLI `--label` arrived in 6.4.0. Those are caller-supplied values. If used, configure `workflow.labels_config.pod_label_prefix: caller.example.com/` in the appropriate configuration wrapper and keep identity templates administrator-owned. Do not use caller labels as proof of who submitted.

OSMO captures retain the original requests and outcomes. These live tests used development authentication headers; production login and NVIDIA username agreement remain unverified.

OSMO: map a known username

A maintained administrator mapping is another existing option. This 6.3.1 ConfigMap-mode fragment shows exactly where the template and platform settings go. Merge it with existing values. Replace `YOUR_POOL` and `YOUR_CPU_PLATFORM`; replace `existing-template` with the platform's current template names, in their current order, then append `attribution`.

```
services:
  configs:
    podTemplates:
      attribution:
        metadata:
          annotations:
            workload.dgxc.nvidia.com/submitter-id: >-
              {{KARTA_USER_BY_SUBMITTER.get(
                WF_SUBMITTED_BY | default(''), {}).get('principal', '')}}
            workload.dgxc.nvidia.com/org_hierarchy: >-
              {{KARTA_USER_BY_SUBMITTER.get(
                WF_SUBMITTED_BY | default(''), {}).get('org', '')}}
    pools:
      YOUR_POOL:
        platforms:
          YOUR_CPU_PLATFORM:
            override_pod_template: [existing-template, attribution]
            default_variables:
              KARTA_USER_BY_SUBMITTER:
                alice:
                  principal: alice@example.test
                  org: demo_research_training
```

For 6.4, put `podTemplates` and `pools` under `configuration`, or inside `configuration.snapshot` when set. In 6.3.1 database mode, register the template with `PUT /api/configs/pod_template/attribution` and patch `PATCH /api/configs/pool/YOUR_POOL` with `configs_dict.platforms.YOUR_CPU_PLATFORM` containing the preserved template list and `default_variables`. These are administrator changes; normal workflow callers should not edit the mapping.

Both `alice` and `alice@example.test` produced `submitted_by="alice@example.test"` on the two Karta metrics. Missing `a` stayed empty. All three groups completed; leaders exited 0, workers had no reported exit code. This requires unique usernames and a maintained map; it cannot recover an ambiguous identity. Organization was a test value. No organization metric was added.

OSMO: authenticated login and native metrics

For an authenticated 6.3.1 gateway, select the username claim on each human JWT provider. Keep existing issuer, audience, JWKS and other providers:

```
gateway:
  envoy:
    jwt:
      user_header: x-osmo-user
      providers:
        - issuer: https://idp.example.test/realms/osmo
          audience: YOUR_CLI_OR_BROWSER_AUDIENCE
          jwks_uri: https://idp.example.test/realms/osmo/certs
          cluster: idp
          user_claim: preferred_username
```

Use a verified, unique IdP claim for browser and CLI tokens. Keep `unique_name` for OSMO-issued tokens. For 6.4 external OIDC, use `authentication.externalOidc.userClaim`. Changing it changes the account identity; existing tokens and ownership do not migrate. This setting was source-reviewed, not tested with a production IdP.

OSMO also already exports the full stored identity in its own metrics:

```
osmo_tasks_count{
  pool="default", status="RUNNING",
  user="alice@example.test",
  workflow_uuid="4c92f554382c4875b9e4add587563721"
} 1.0
```

In 6.3.1, keep `services.service.disableTaskMetrics: false`. In the unified 6.4 chart, use `services.api.disableTaskMetrics: false`. Scrape port 9464, `/metrics`; this was enabled and verified in the 6.3.1 lab. The gauge counts tasks; completed-task coverage is short. A UUID join supports reporting, but does not populate Karta's metadata-only CEL input.

OSMO already has workflow/group UUIDs and KAI PodGroups. Karta currently follows controller ownership, so these bare task Pods remain individual workloads. That grouping gap belongs to Karta. The OSMO request is for lossless principal access. Source audit and retained captures document the settings and test limits.

NVCF / NVCT: reproduce the function

The container function served HTTP 200 and the finite task completed. Both carried account and workload IDs, but neither placed the authenticated caller in Kubernetes metadata. Both tested Karta exporter PoC metric families kept `submitted_by=""`.

Use your management URL/token as `NVCF_API` / `NVCF_ADMIN_TOKEN`, and an available `GPU` / `INSTANCE_TYPE`. The management credential needs `register_function`, `deploy_function`, `delete_function` and either `list_functions` or `list_functions_details`. The invocation key needs `invoke_function`; API-key authorities have the `apikey:` prefix. These commands contain the complete HTTP fixture:

```
curl -fsS "$NVCF_API/v2/nvcf/functions" \
  -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" \
  -H 'Content-Type: application/json' --data-binary @- \
  > function.json <<'JSON'
{
  "name":"cel-label-function",
  "containerImage":"nvcr.io/nvidia/omniverse/simple-nginx:1.0.2",
  "inferenceUrl":"/", "inferencePort":80,
  "health":{"protocol":"HTTP","uri":"/","port":80,
            "timeout":"PT10S","expectedStatusCode":200}
}
JSON
FID=$(jq -r '.function.id' function.json)
VID=$(jq -r '.function.versionId' function.json)
DEPLOY="$NVCF_API/v2/nvcf/deployments/functions/$FID/versions/$VID"
jq -n --arg gpu "$GPU" --arg it "$INSTANCE_TYPE" \
  '{deploymentSpecifications:[{gpu:$gpu,instanceType:$it,
    minInstances:1,maxInstances:1,maxRequestConcurrency:1}]}' \
  > deploy.json
curl -fsS "$DEPLOY" -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" \
  -H 'Content-Type: application/json' --data-binary @deploy.json
```

Check the function version until ACTIVE, then invoke it using your invocation URL/domain and key. The tested gateway routes by the function Host header:

```
curl -fsS "$NVCF_API/v2/nvcf/functions/$FID/versions/$VID" \
  -H "Authorization: Bearer $NVCF_ADMIN_TOKEN"
curl -fsS "$NVCF_INVOKE_URL/" \
  -H "Host: $FID.$NVCF_INVOKE_DOMAIN" \
  -H "Authorization: Bearer $NVCF_API_KEY"
# Release this function's allocation before the finite task.
curl -fsS -X DELETE "$DEPLOY" \
  -H "Authorization: Bearer $NVCF_ADMIN_TOKEN"
```

Observed: ACTIVE, HTTP 200. The follow-up added `{"key":"KARTA_TEST_SUBMITTED_BY","value":"alice@example.test"}` to `containerEnvironment`. It reached the function's Pod spec, but no annotation or label. The image had no shell, so its process environment was not inspected.

Lab versions: NVCF `1.18.0`, NVCT `1.66.0`, NVCA `3.10.0`, ReVal `0.20.2`. The lab used service credentials, fake GPU allocation and an HTTP fixture. The two test Pods needed a lab-only CPU resize to 100m; the API requests do not apply that adjustment. Human login, hosted NVCF and model inference were not tested.

Original requests and outcomes and environment follow-up.

NVCT: reproduce a task that finishes

Use `NVCT_API` / `NVCT_API_KEY` with `launch_task` and `task_details` permissions. A bare sleep does not complete the NVCT protocol: write progress, reach 100, then exit. This script reproduces the saved finite task, with result uploads disabled:

```
cat > nvct-task.sh <<'SH'
set -eu
for attempt in $(seq 1 60); do
  [ -d "$NVCT_RESULTS_DIR" ] && break
  sleep 1
done
[ -d "$NVCT_RESULTS_DIR" ]
sleep 1
progress() {
  printf '{"taskId":"%s","percentComplete":%s,"name":"%s","lastUpdatedAt":"%s"}\n' \
    "$NVCT_TASK_ID" "$1" "$2" "$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
    > "$NVCT_PROGRESS_FILE_PATH.tmp"
  mv "$NVCT_PROGRESS_FILE_PATH.tmp" "$NVCT_PROGRESS_FILE_PATH"
}
progress 1 running
sleep 180
progress 100 completed
sleep 10
SH
jq -n --arg gpu "$GPU" --arg it "$INSTANCE_TYPE" \
  --arg script "$(base64 < nvct-task.sh | tr -d '\n')" '{
  name:"cel-label-task", gpuSpecification:{gpu:$gpu,instanceType:$it},
  containerImage:"nvcr.io/nvidia/cuda:12.6.3-base-ubuntu22.04",
  containerArgs:"bash -c '\''printf %s \"$ATTRIBUTION_SCRIPT\" | base64 -d | bash'\''",
  maxRuntimeDuration:"PT5M", maxQueuedDuration:"PT10M",
  terminationGracePeriodDuration:"PT15S", resultHandlingStrategy:"NONE",
  containerEnvironment:[{key:"ATTRIBUTION_SCRIPT",value:$script}]
}' > task.json
curl -fsS "$NVCT_API/v1/nvct/tasks" \
  -H "Authorization: Bearer $NVCT_API_KEY" \
  -H 'Content-Type: application/json' --data-binary @task.json > task-result.json
TASK_ID=$(jq -r '.task.id' task-result.json)
curl -fsS "$NVCT_API/v1/nvct/tasks/$TASK_ID" \
  -H "Authorization: Bearer $NVCT_API_KEY"
```

Observed: COMPLETED, 100%. The follow-up environment value also reached `printenv` in the task. Neither test projected it into metadata.

The lab needed these helper-image overrides; the public image references were confirmed in its current ConfigMap. Merge these settings into the existing values, preserving other sidecars:

```
nvctApi:
  remoteConfig:
    enabled: true
    configData:
      nvct:
        sidecars:
          init-container: "nvcr.io/nvidia/nvcf/nvcf-worker-init-oss:1.2.1"
          utils-container: "nvcr.io/nvidia/nvcf/nvcf-worker-task-oss:1.1.4"
```

These are the actual chart setting names. The application properties are `nvct.sidecars.init-container` and `nvct.sidecars.utils-container`. Earlier malformed-task cancellation left an ICMSRequest requiring manual cleanup. This and helper-image packaging are separate from the identity request; suggest separate issues if reproduced on the target release.

NVCF Helm: configure the test

The live Helm function reached ACTIVE. Its Service returned HTTP 200, and both Karta metrics carried `submitted_by="transport-test-only"`. NVCF's existing NGC registry credential pulled the chart. No registry credential change or image/chart push was needed.

Use the same management token and available GPU/instance type as the container test. This uses NIM chart `1.14.0-pb5.0` to launch a small HTTP server. It tests Helm metadata, not NIM inference. Save the complete chart values:

```
cat > helm-values.json <<'JSON'
{
  "fullnameOverride":"karta-helm-attribution-retest",
  "statefulSet":{"enabled":false}, "replicaCount":1,
  "image":{
    "repository":"docker.io/library/python",
    "tag":"3.12-alpine@sha256:1b668429b3511ab407d8e00648891631b0b1a4d7e15e3ca70f38ab5b91ad4ab4",
    "pullPolicy":"IfNotPresent"
  },
  "customCommand":["python3","-m","http.server","8080","--directory","/tmp"],
  "env":[{"name":"PYTHONDONTWRITEBYTECODE","value":"1"}],
  "imagePullSecrets":[{"name":"karta-http-fixture-unused-pull"}],
  "model":{"ngcAPIKey":"unused-http-fixture",
           "ngcAPISecret":"karta-http-fixture-placeholder","openaiPort":8080},
  "podAnnotations":{
    "workload.dgxc.nvidia.com/submitter-id":"transport-test-only"
  },
  "podSecurityContext":{"runAsUser":1001,"runAsGroup":1001,"fsGroup":1001},
  "containerSecurityContext":{"allowPrivilegeEscalation":false,
    "capabilities":{"drop":["ALL"]},"runAsNonRoot":true},
  "resources":{"requests":{"cpu":"50m","memory":"64Mi"},
    "limits":{"cpu":"250m","memory":"128Mi","nvidia.com/gpu":1}},
  "service":{"type":"ClusterIP","name":"karta-helm-attribution-retest",
             "openaiPort":8080},
  "startupProbe":{"path":"/","initialDelaySeconds":0},
  "readinessProbe":{"path":"/","initialDelaySeconds":0},
  "livenessProbe":{"path":"/"},
  "serviceAccount":{"create":false}, "persistence":{"enabled":false},
  "multiNode":{"enabled":false}, "ingress":{"enabled":false}
}
JSON
```

`unused-http-fixture` is a noncredential placeholder for chart-generated Secrets. This public HTTP image does not use it or download a model. Real model deployments need their normal credentials.

Required lab settings:

- On NVCA 3.10.0, ReVal used its default Kubernetes v1.20.0 because the actual server version was not supplied. The chart needs at least v1.23.0. Set the operator chart's `k8sVersionOverride` to the actual server version, `v1.35.5` in this lab. Current public source includes automatic version discovery; a release containing it was not tested.
- The lab had no SMB CSI driver. For this storage-free function, append `-HelmSharedStorage` to the NVCA operator chart's existing `worker.featureFlags` list. This affects the whole backend; preserve other flags and restore it after testing. Helm tasks and functions needing shared storage require the normal storage setup.
- The lab already used `reval.serviceConfig.skipSanitizeObjectMetadata: true`. Stricter sanitization was not live-tested. With sanitization enabled, exact `preserveAnnotations` keys preserve root annotations; this chart's `podAnnotations` targets the Deployment template.

NVCF Helm: submit and inspect the result

```
CHART=https://helm.ngc.nvidia.com/nvidia/charts/nim-llm-1.14.0-pb5.0.tgz
jq -n --arg chart "$CHART" '{
  name:"karta-helm-attribution-retest-20261016", helmChart:$chart,
  helmChartServiceName:"karta-helm-attribution-retest",
  inferenceUrl:"/", inferencePort:8080, healthUri:"/", apiBodyFormat:"CUSTOM"
}' | curl -fsS "$NVCF_API/v2/nvcf/functions" \
  -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" \
  -H 'Content-Type: application/json' --data-binary @- > helm-function.json
HFID=$(jq -r '.function.id' helm-function.json)
HVID=$(jq -r '.function.versionId' helm-function.json)
HDEPLOY="$NVCF_API/v2/nvcf/deployments/functions/$HFID/versions/$HVID"
HFUNCTION="$NVCF_API/v2/nvcf/functions/$HFID/versions/$HVID"
jq -n --arg gpu "$GPU" --arg it "$INSTANCE_TYPE" \
  --slurpfile values helm-values.json \
  '{deploymentSpecifications:[{gpu:$gpu,instanceType:$it,
    minInstances:1,maxInstances:1,maxRequestConcurrency:1,
    configuration:$values[0]}]}' |
  curl -fsS "$HDEPLOY" -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" \
    -H 'Content-Type: application/json' --data-binary @-
curl -fsS "$HFUNCTION" -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" |
  jq '.function.status'
```

Wait for ACTIVE. Find the generated namespace with `kubectl get deployments -A`, then set `HELM_NS`. Using cluster credentials with read and port-forward access:

```
kubectl -n "$HELM_NS" get deployment karta-helm-attribution-retest \
  -o json | jq '.spec.template.metadata.annotations'
kubectl -n "$HELM_NS" port-forward \
  svc/karta-helm-attribution-retest 18080:8080
# In another terminal:
curl -i http://127.0.0.1:18080/
```

Observed: HTTP 200. The annotation was on the Deployment's Pod template and its Pod, not on the Deployment root. Karta followed Pod -> ReplicaSet -> Deployment and used the Pod identity for both metric families. The first page's CEL already reads this key. Selected labels:

```
karta_pod_workload_info{workload="karta-helm-attribution-retest",
  workload_kind="Deployment", submitted_by="transport-test-only"} 1
karta_workload_status{workload="karta-helm-attribution-retest",
  phase="Running", submitted_by="transport-test-only"} 1
karta_exporter_observation_healthy{workload="karta-helm-attribution-retest",
  scope="workload", reason="none"} 1
```

The HTTP check used the Service port-forward. A separate gateway attempt with the management token returned 403; it did not prove gateway invocation for this Helm function. The container-function gateway test above used its invocation key and passed.

After capture, stop the port-forward and remove only this test function:

```
curl -fsS -X DELETE "$HDEPLOY" -H "Authorization: Bearer $NVCF_ADMIN_TOKEN"
curl -fsS -X DELETE "$HFUNCTION" -H "Authorization: Bearer $NVCF_ADMIN_TOKEN"
```

The lab test was removed and both temporary NVCA settings restored. Two failed attempts left ICMSRequests that required manual cleanup. Retained requests, metadata, metrics and cleanup.

NVCF / NVCT: what remains

Helm metadata transport now works end to end. `transport-test-only` was supplied in chart values; NVCF did not derive it from an authenticated user. The direct-container function and task still had no submitter metadata. No Karta, NVCF or OSMO source/API change was needed for these tests.

Existing options:

- Audit joins: collect INFO-level `[AUDIT]` events across NVCF/NVCT replicas. Join account plus function/version/deployment or task ID, preserving operation and time. Events include authenticated actor/subject identity. This needs a log pipeline and identity mapping; it does not feed Karta's metadata-only CEL. Audit emission.
- Fixed classification: NVCA `agent.customAnnotations` is copied literally onto directly created worker Pods. It can identify a dedicated backend's organization, but cannot derive each submitter. `HelmCustomAnnotations` separately controls the Helm utility Pod, not every chart Pod.
- Trusted submission service: an existing authenticated service could derive the principal and supply the Helm annotation. It must control the submission path and prevent callers from replacing the authoritative value. This additional component was not implemented or tested here. Arbitrary client values are not proof of identity.

The preferred native solution is to retain the authenticated function creator, deployment actor and task submitter, then propagate the appropriate principal to workload/Pod annotations. A shared service credential must remain a service identity unless verified human delegation exists. Function invocation is a separate action and should not rewrite workload identity.

The NVCF request draft is limited to that capture and propagation. Generic Helm values, account IDs and audit logs already exist.

Grouping is also separate. The Helm test resolved one Deployment workload. In the earlier direct-container captures, Pod owner references lacked `controller: true`, so the tested Karta exporter selected each Pod. OSMO bare tasks have the same integration gap. Their existing workload IDs and grouping metadata are available for a future Karta integration.

Confirm the result and track the remaining gaps

Submit the fixtures above with your own platform login. While Pods still exist, run these instant queries in Prometheus or Grafana Explore, replacing `YOUR_NAMESPACE`:

```
karta_workload_status{namespace="YOUR_NAMESPACE"} == 1
karta_pod_workload_info{namespace="YOUR_NAMESPACE"}
karta_exporter_observation_healthy{
  namespace="YOUR_NAMESPACE", scope="workload"
}
```

The first query shows the active phase with `submitted_by`. The second shows each Pod's workload and submitter. The third should be 1 for a healthy workload observation. An empty identity means unknown. Missing rows are not proof of success: check exporter scrape health, namespace scope and read permissions. Completed workloads can disappear after platform cleanup; use the recorded time range to inspect their earlier samples.

Path	Confirmed	Remaining gap / next step
Direct Kubernetes	Authenticated caller captured; forged create and identity update rejected.	Applies only inside admission scope. Service accounts do not establish a separate human.
Run:AI	Training completed; inference HTTP 200; both attributed from workload metadata.	Agree the NVIDIA username format. External ingress and model inference were not tested.
OSMO	Native label, guarded template and administrator mapping reach Karta. Full user is also in OSMO task metrics.	Production IdP not tested. Request lossless principal access; map only unique usernames in the meantime.
NVCF / NVCT	HTTP function and finite task work. Supplied environment values reach containers.	Human metadata remains empty. Audit joins exist; request trusted creator/deployer/submitter metadata.
NVCF Helm	ACTIVE; Service HTTP 200; Pod annotation appears on both Karta metrics.	Supplied value is not authenticated identity. Use a trusted submission path or native capture. Helm gateway invocation was not verified.
Karta grouping	Controller ownership works for tested Kubernetes and Run:AI paths.	OSMO bare tasks and captured NVCF direct Pods remain separate workloads. Add integration for their existing grouping metadata.
Karta selectors	Admission namespace selectors and CEL identity matching work.	Admission GVK `matchConditions` and workload LIST/WATCH selectors are proposals.

These results cover the shown versions and paths. They do not establish all platform submission modes, production identity mapping or 100,000-workload attribution performance. The OSMO and NVCF request drafts are ready for discussion; no issues were opened.